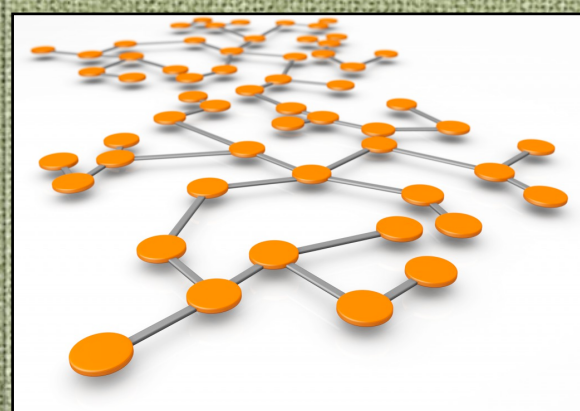




Sistemas Distribuídos

Professor: Rodrigo da Rosa Righi

Contato: rrrighi@unisinos.br

Aula: 2

Dia/Horário: Sexta-Feira, 19:30 - 22:23

Agenda

- * Parâmetros de Desempenho
- * Comunicação Interprocessos
 - * Comunicação Síncrona e Assíncrona
 - * Destino das Mensagens
 - * Comunicação One-Sided
- * Bibliotecas de Programação para Passagem de Mensagens
- * Programando com Sockets
 - * Datagrama UDP
 - * Stream TCP



Você se lembra da Aula 1?

**O que é um Sistema
Distribuído**

**Organização de um Sistema
Distribuído**

**Arquiteturas Comuns e
Avançadas de Sistemas
Distribuídos**



Material Complementar

Desafios de Sistemas
Distribuídos

<http://mathieu.delalandre.free.fr/teachings/distributed/part1.pdf>

Artigo sobre Sistemas
Distribuídos
Altamente Complexos

[http://www.google.com.br/url?
sa=t&rct=j&q=&esrc=s&source=web&cd=2&cad=rja&uact=8&ve
d=0CDgQFjAB&url=http%3A%2F%2Fwww.dre.vanderbilt.edu%2
F~schmidt%2FPDF%2FFOME-HCDS-
paper.pdf&ei=qbMYU7zGCaie2wX7jYCoDA&usg=AFQjCNH9B
QMwZ9jaMuAyCQ02x5xV_J7uoA&sig2=MtUE5loR6C3FdYc_y
1W8Zg&bvm=bv.62577051,d.b2I](http://www.google.com.br/url?sa=t&rct=j&q=&esrc=s&source=web&cd=2&cad=rja&uact=8&ved=0CDgQFjAB&url=http%3A%2F%2Fwww.dre.vanderbilt.edu%2F~schmidt%2FPDF%2FFOME-HCDS-paper.pdf&ei=qbMYU7zGCaie2wX7jYCoDA&usg=AFQjCNH9BQMwZ9jaMuAyCQ02x5xV_J7uoA&sig2=MtUE5loR6C3FdYc_y1W8Zg&bvm=bv.62577051,d.b2I)

Material Complementar

Armadilhas de Sistemas Distribuídos (*pitfalls*)

A rede é confiável

Existe sempre um administrador

A rede é segura

Custo de transporte é zero

A rede é homogênea

A largura de banda é infinita

A topologia de rede não muda

A latência de rede é zero

Parâmetros de Desempenho

Desempenho

Latência

Largura de banda

Jitter

Parâmetros de Desempenho

O atraso entre o começo da transmissão de uma mensagem do processo origem até a sua recepção pela aplicação no destino

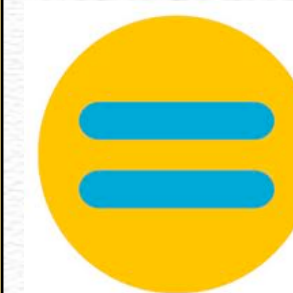
Serviços de
comunicação



Tempo para
acessar a rede

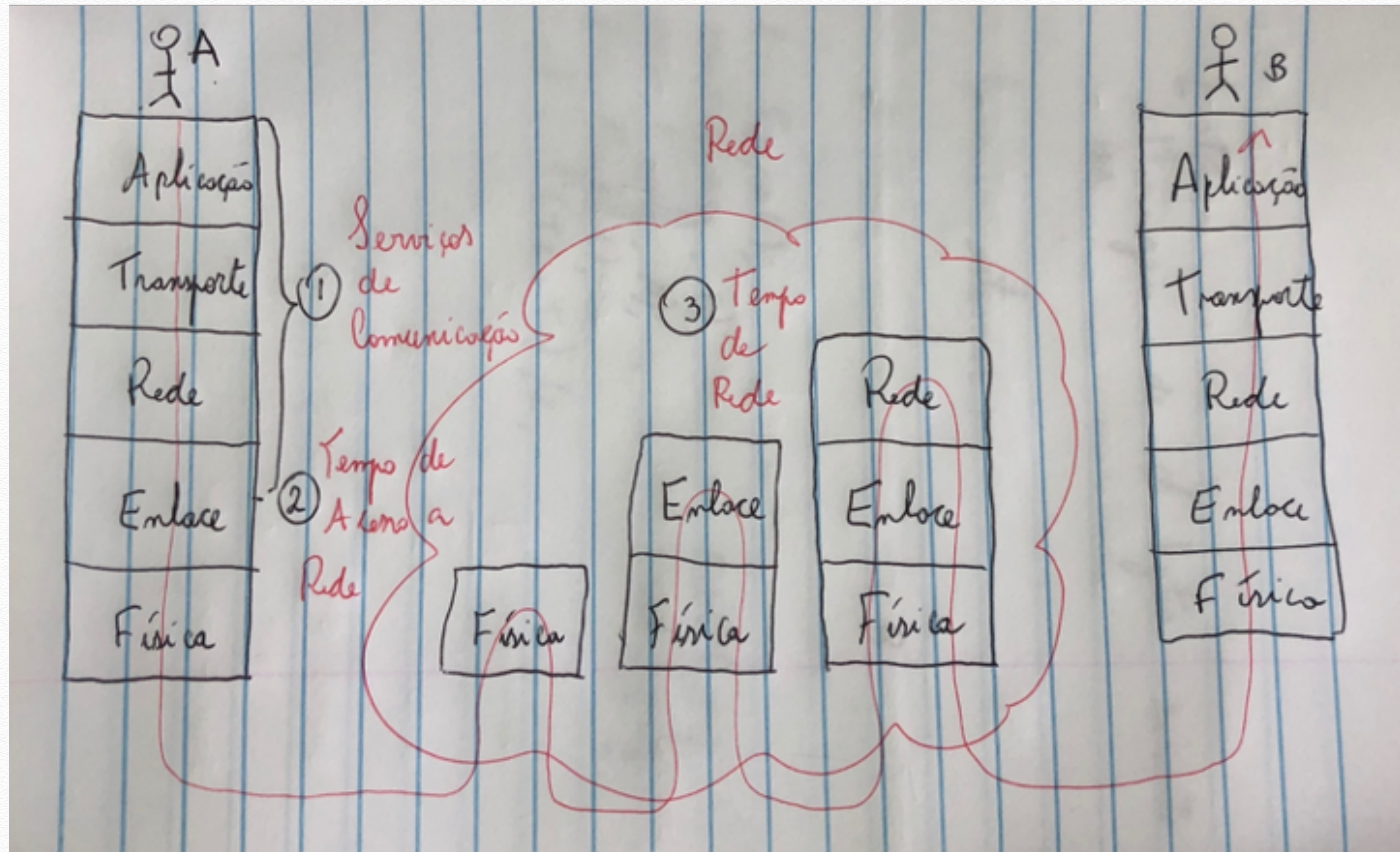


Tempo de
Rede



Latência

Latência



Latência

Latência -
Mensagem de ϕ bytes - somente
IDA.

Como estimar:

A

B

$t_1 = \text{tempo}();$

$\text{send}(\text{msg}, B);$

$\text{recv}(\text{msg}, A);$

$t_2 = \text{tempo}();$

$\text{Lat} = (T_2 - T_1) / 2;$

$\text{recv}(\text{msg}, A);$

$\text{send}(\text{msg}, B);$

Parâmetros de Desempenho

Total de informações que podem ser transmitidos em um dado tempo

Quando várias estações compartilham a mesma rede, eles devem compartilhar largura de banda

Normalmente medido em bits por segundo

Largura de Banda

Parâmetros de Desempenho

Variação no tempo tomado para entregar uma série de mensagens

Atraso entre pacotes sucessivos de dados

Por exemplo, amostras consecutivas de dados de audio são disparadas com diferentes intervalos de tempo, então o som será distorcido.

Sistemas
Multimedia

Uma das formas de minimizar a variação de atraso do Jitter é a utilização de buffer, o qual armazena os dados à medida que eles chegam e os encaminha para a aplicação segundo uma mesma cadência.

Jitter

Redes e Sistemas Distribuídos

Latência



Tempo para Transferir
uma mensagem VAZIA

Tempo de
Transferência



Latência



Tamanho da
Mensagem

Largura de Banda

Taxa de
Transferência



Determinada primeiramente pelas características físicas, onde a latência é basicamente determinada pela sobrecarga de software, atrasos de roteamento e a carga do canal de comunicação.

Redes e Sistemas Distribuídos

Muitas mensagens transferidas em sistemas distribuídos são de tamanho pequeno

Latência possui muitas vezes **IGUAL** ou **MAIOR** significância que a taxa de transferência para determinar o desempenho do sistema.

Comunicação Interprocessos

Passagem de Mensagens

Duas diretivas
de
comunicação

`send()`

`receive()`

Pairwise
Directives

Podemos ter sincronização entre
processos

Comunicação
Síncrona

Comunicação
Assíncrona

Comunicação Interprocessos

Comunicação Síncrona

Analogia com
uma fila

Processo que transmite faz com que os dados sejam adicionados na fila de recepção

Processos receptores removem mensagens de sua fila local

Bloqueantes

Envio

Recepção

Processos se sincronizam a cada mensagem trocada

Comunicação Interprocessos

Comunicação Síncrona

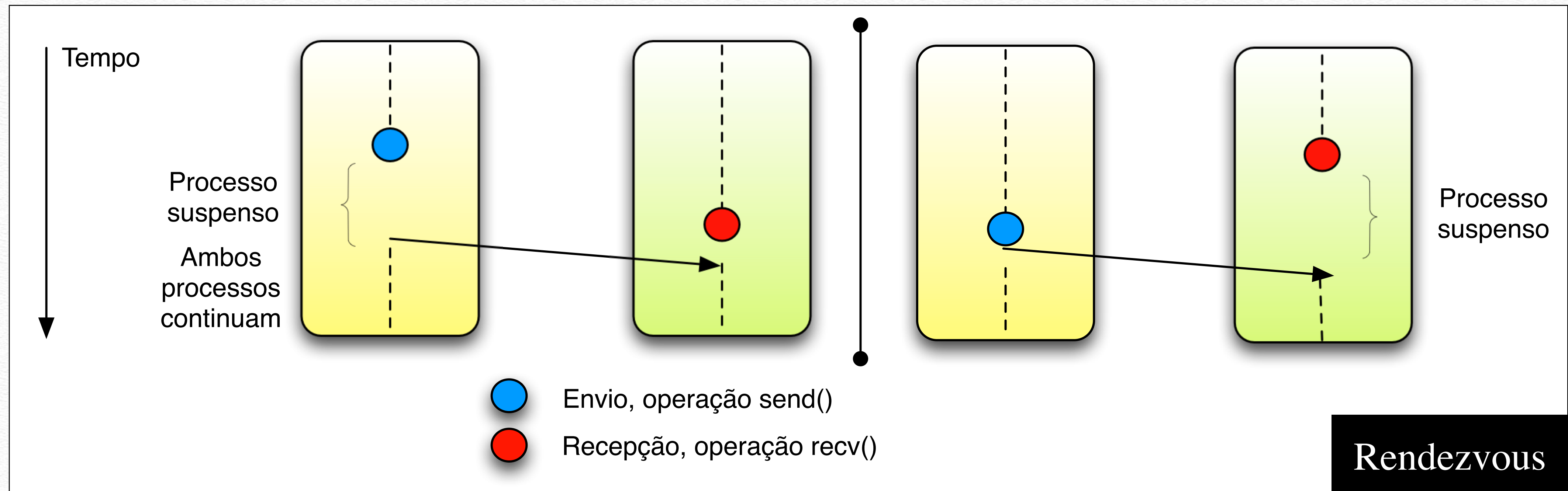
Lançamento

Transmissor

Receptor

Ele é bloqueado até que uma mensagem correspondente seja lançada

Processo espera até que uma mensagem chegue em sua fila



Comunicação Interprocessos

Comunicação Assíncrona

Transmissão

Recepção

Não Bloqueante

Não Bloqueante

Bloqueante

Processo tem a permissão de proceder tão logo a operação tenha copiado os dados para um buffer local

Receptor procede e há um buffer a ser preenchido em background

Programa receptor verifica Buffer

Processo espera pelos dados em sua fila de recepção

Não
Bloqueante

Polling

Chamada de Teste Bloqueante

Interrupção

Tratador

Comunicação Interprocessos

Ideia

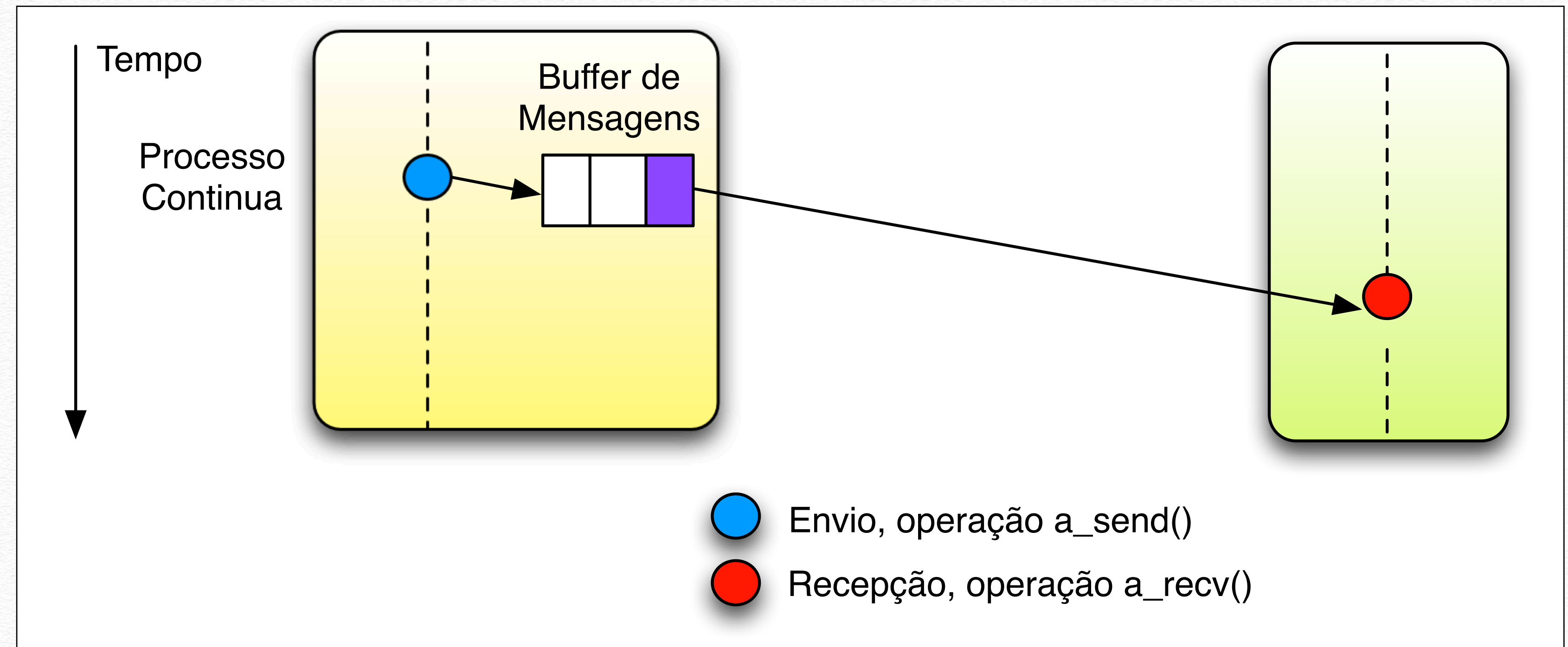
Custo

Maioria dos sistema
oferece recepção
bloqueante

Eficiência

Complexidade

Comunicação
Assíncrona



Comunicação Interprocessos

Comunicação Assíncrona

É possível
prover
assincronismo
com diretivas
síncronas?

Transmissor pode ter duas threads: uma produtora de mensagens no buffer e outra consumidora de mensagens no buffer.

Problema produtor-consumidor onde buffer é uma região crítica pois representa uma memória compartilhada entre threads.

Utilização de variáveis de exclusão mútua e de sinalização (wait e notify).

A thread consumidora realiza de fato a comunicação dos dados pela rede para o processo receptor.

Comunicação Interprocessos

Comunicação Assíncrona

Esconder a latência de comunicação

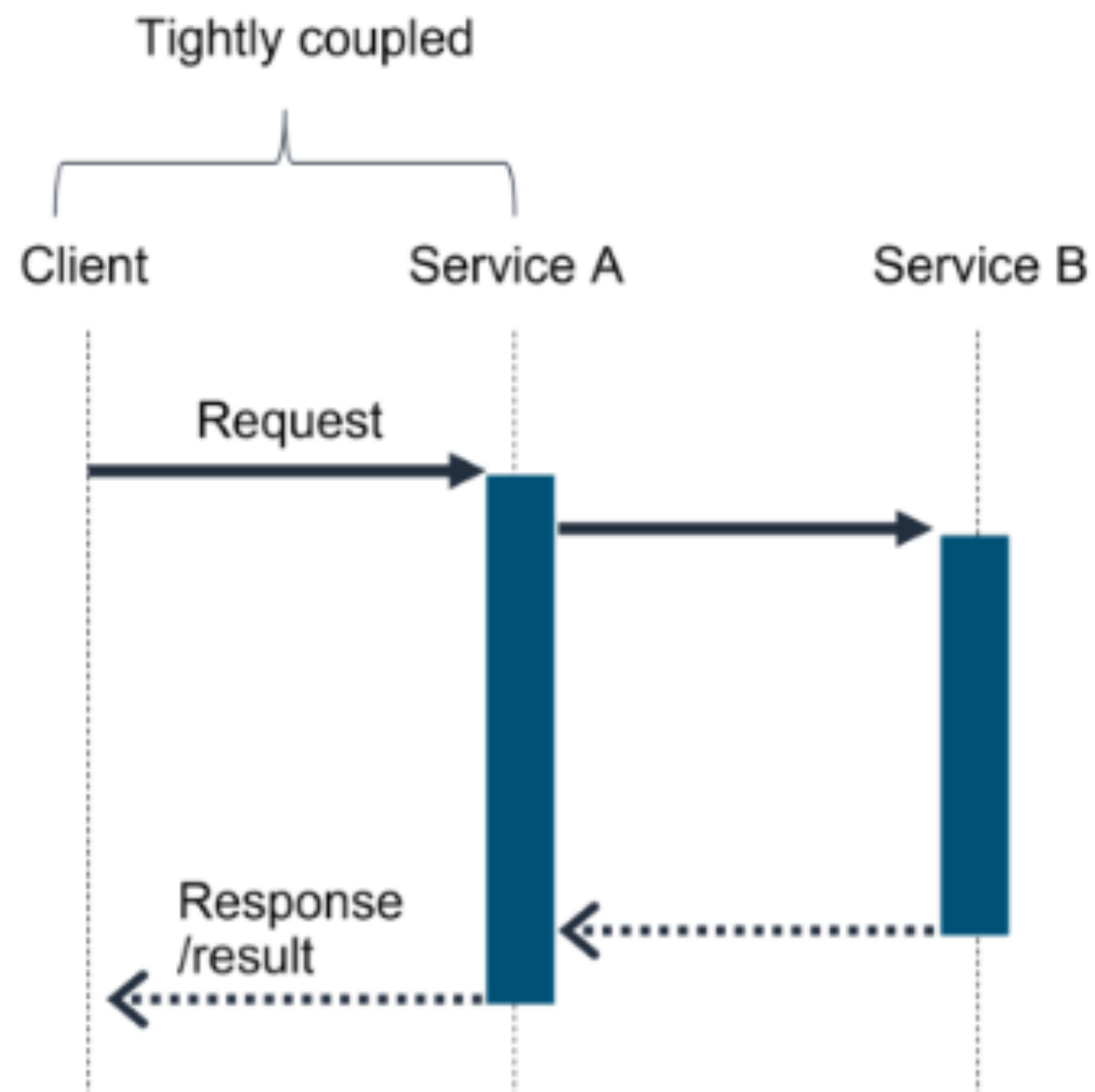
Transmissão de dados ocorre em paralelo
com computação

Largamente usada em Processamento de
Alto Desempenho

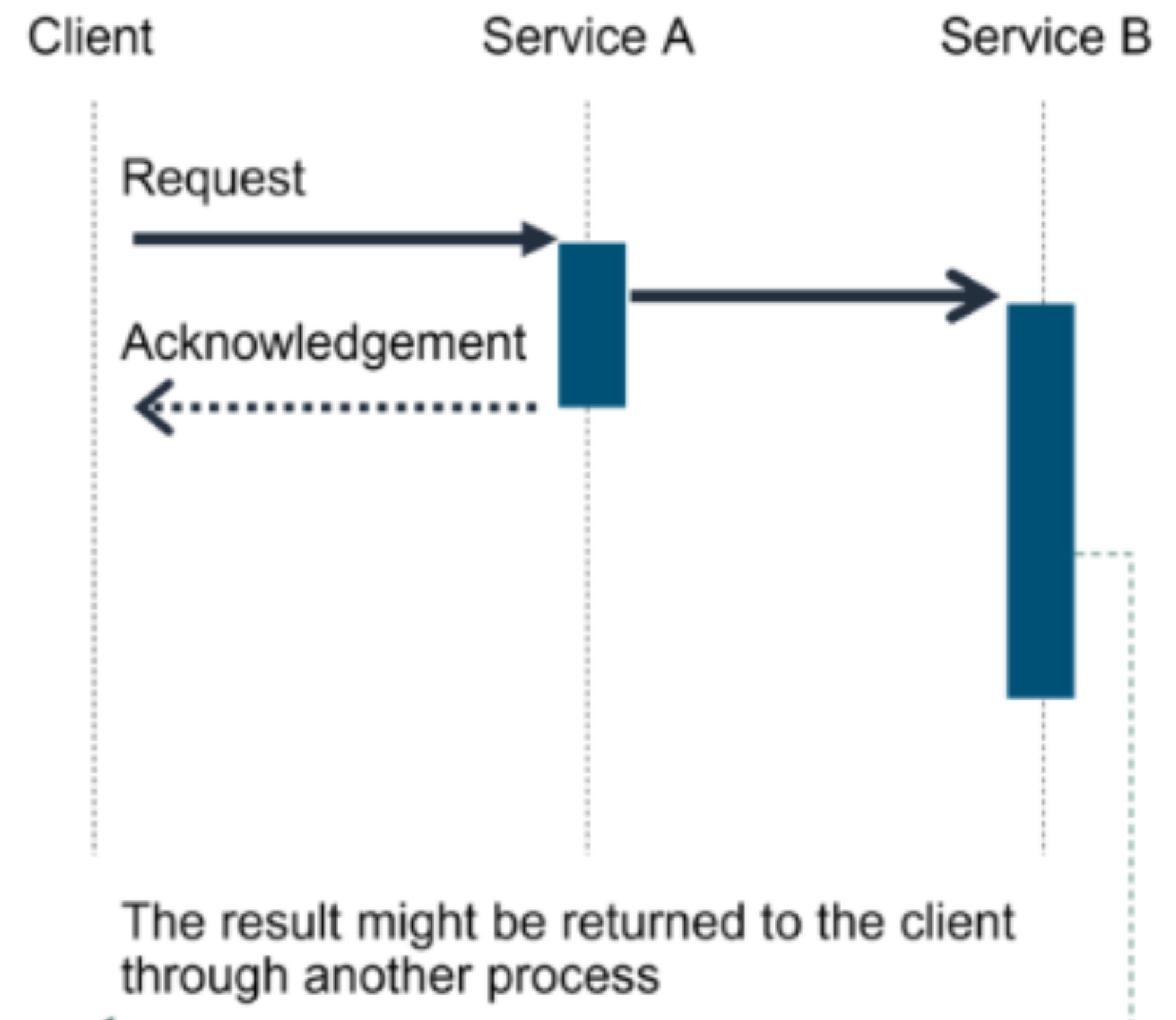
Ganho

Comunicação Interprocessos

Comunicação Síncrona



Comunicação Assíncrona



Comunicação Interprocessos

Destino das Mensagens

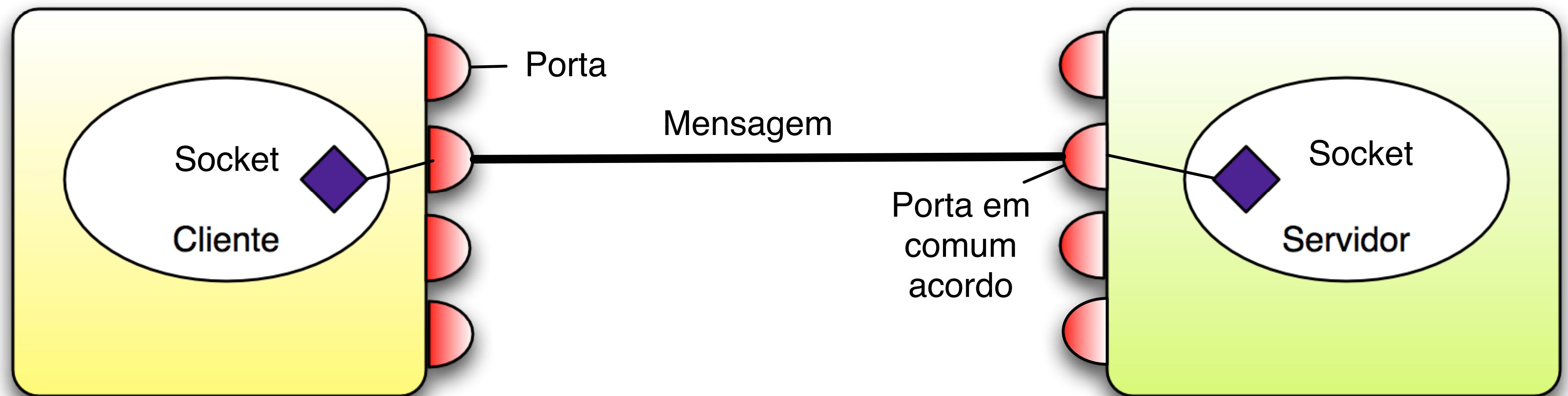
Mensagens enviadas para um PAR

Endereço IP

Porta

Processos podem
usar múltiplas
portas

Servidores tornam
público o número de suas
portas para os clientes



Endereço de Internet = 62.233.163.104

Endereço de Internet = 62.233.163.105

Comunicação Interprocessos

Comunicação One-Sided

Bibliotecas recentes possuem
diretivas One-sided

Ideia

Complexidade

Processos registram
uma região de
memória local e
passam o tratador de
memória para o
objeto remoto

Outros
processos
podem então ler
e escrever na
memória do
processo
remoto

Não há sincronização

Comunicação Interprocessos

Comunicação
One-Sided

Onde é
encontrado

Bibliotecas de comunicação para redes de alta velocidade, como Myrinet, Quadrics e Infiniband

Operações RDMA (Remote Direct Memory Access)

Transferência sem trocas de contexto e sem cópias intermediárias

Sockets

Ponto Final para
comunicação entre
processos

Origem: BSP
Unix

Vários processos podem enviar
mensagens para a mesma porta

Cada Socket está associado a
um protocolo de transporte

UDP

TCP

Sockets

Sockets UDP

Um datagrama UDP é enviado de um processo e recebido por outro sem ACK (Acknowledgement)

Sockets UDP normalmente fornecem envio não bloqueante e recepção bloqueante

Se uma falha ocorrer, a mensagem pode não chegar

Recepção de mensagem de qualquer origem

Uso

DNS

Multimedia em geral

SNMP

DHCP

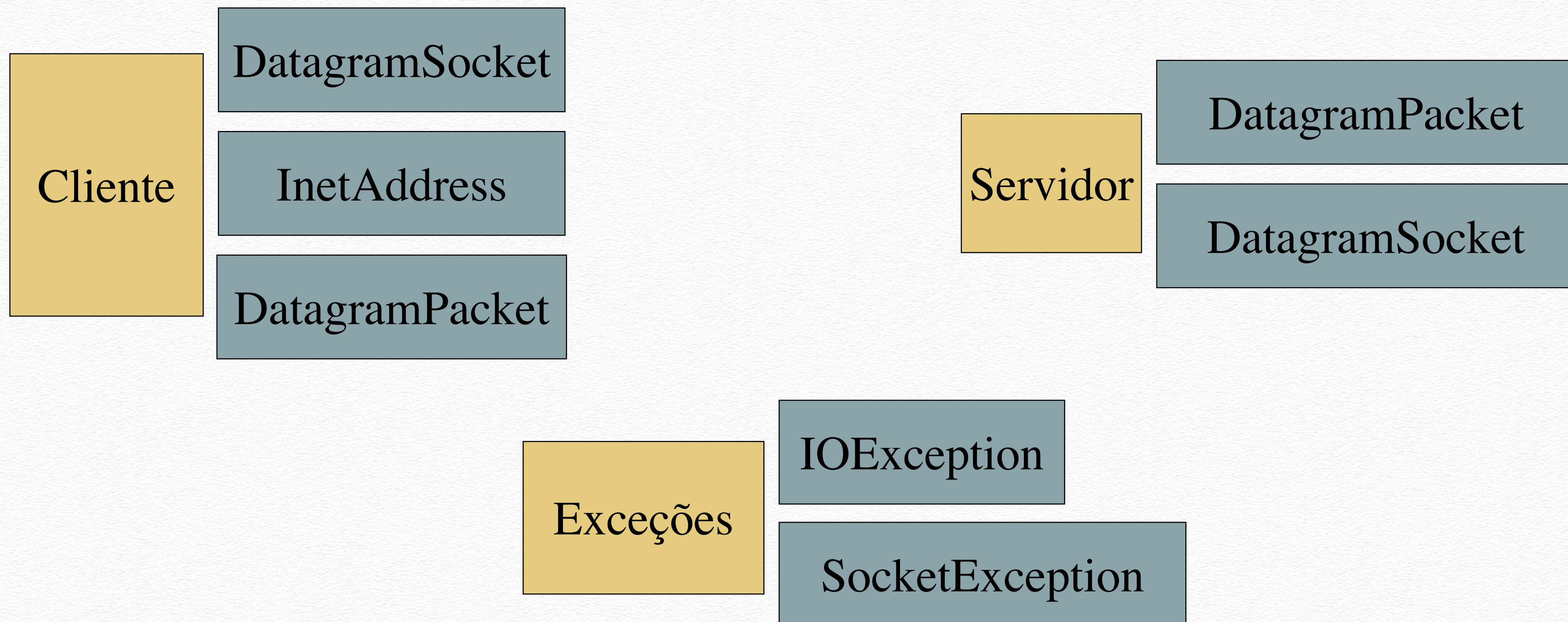
Aplicações que aceitam falhas ocasionais

Desempenho



Sockets

Sockets UDP



Sockets

Sockets TCP

Streams

Tamanho da
mensagem

Mensagens pequenas e grandes

Flush

Mensagens
perdidas

Opera com ACK

Grava cada segmento enviado de modo a receber o ACK

Retransmissão baseada em timeout

Controle de
Fluxo

TCP tenta mapear velocidades dos processos
que leem e escrevem em um stream

Ideia é “segurar” o transmissor

Sockets

Sockets TCP

Destino das
mensagens

Uma vez que a conexão foi estabelecida,
pode-se usar `send()` e `receive()`

Requisição de conexão pelo cliente
(`connect`) e aceite no servidor (`accept`)

Overhead

TCP é uma das maiores fontes de overhead na comunicação

Casamento de
itens de dados

Processos trocam bytes

Um processo pode mandar um “int” e um “double”.
Receptor deve ler na mesma ordem.

Sockets

Sockets TCP

Bloqueante

Processo deve esperar pela operação complementar de troca de mensagens

Ideia de sincronismo

Fluxos de execução

Quando um servidor aceita uma conexão, ele geralmente cria uma nova thread para se comunicar com o novo cliente.

Implementação

Quando threads não são fornecidas, uma alternativa é testar se uma entrada está disponível em um stream antes de lê-lo.

`select()`

Sockets

Sockets TCP

Uso

HTTP

FTP

Telnet

SMTP

Confiabilidade

Integridade dos
dados é requerida

Sockets

Sockets TCP

